

Python for QA

Readable programming language with a strong ecosystem for automation, APIs, data and utilities.

Why QA uses Python

Python is used for Web/API automation, scripts, test-data generation, data analysis and tool integration.

Official site and documentation

Use the official Python for QA source as the primary reference: <https://www.python.org/downloads/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Virtual environment

Use `python -m venv .venv` to isolate dependencies. In PowerShell activate with `.\venv\Scripts\Activate.ps1`.

pip and requirements

`pip` installs packages. `requirements.txt` records reproducible dependencies; install with `pip install -r requirements.txt`.

Fundamentals

Learn variables, types, conditions, loops, functions, collections, modules, exceptions and OOP as needed.

QA in practice

Combine Python with Pytest, Playwright, requests and other libraries while keeping secrets outside code.

How to start safely

Start from the official Python for QA site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in Python for QA, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Python for QA fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether Python for QA still delivers value as the product evolves.

Common issues and troubleshooting

When Python for QA fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.

Installation overview

Use the official Python installer for Windows or the mechanism recommended by current Python documentation. Pay attention to options that make Python available in the terminal. Open a new terminal and validate `python --version` and `py --version`.

pip and virtual environment

Validate packages with `python -m pip --version`. For projects prefer a virtual environment: `python -m venv .venv`, then activate it before installing dependencies.

Forgot PATH

Try `py --version` first. If Python is installed but `python` is not recognized, locate the installation and add the directory containing `python.exe` and, when needed, `Scripts` to `Path`. Reopen the terminal.

First script

Create `hello.py` with `print("QA Lab Brasil")` and run `python hello.py`; `py hello.py` may also be used on Windows.

Troubleshooting

If `pip` targets another Python, prefer `python -m pip`. With multiple versions use `where python` and `py -0p` when available.